Project Report – B100 Python Programming

Introduction

This project implements a simple Library or Bookstore System that manages books, users and borrowing records, which could be said that those are the main functionalities that are needed from such system, and therefore, it targets mainly small bookstore businesses or libraries.

It provides a command-line based software solution using Python and CSV for persistent storage, therefore it can replace manual tracking methods, as it can register and manage books and users, store borrowing information by tracking active and overdue borrowing records as well as store data through CSV files, therefore even when the program is disconnected, the input data will not be lost. The focus of this project was mainly on Object-oriented programming (OOP) in Python and the application of it´s concepts in an effective way. I chose this project specifically because it is well documented, I have found a lot of resources that helped me along the way.

Link to the GitHub repository that has the project files:
https://github.com/Marculate/Simple-Bookstore-Library-System

System Design

During the programming of this application, simplicity and readability of the code was put in mind, therefore, it is a class-based structure, where each has one clear role, as the **main structure** includes the following:

- *main.py* file, which handles menus design and user inputs.
- *Library.py* file, which contains the main core logic of adding books, users, borrow and returns as well as saving and loading date.
- *Books.py*, *Users.py* and *Borrowing_Records.py* which contain the core data models of the system. Those files define the behaviour of the books, users and transactions related to borrowing, as it can be seen in the source code that they include validations for inputs, status as well as conversions to/from CSV-friendly formats.
- *File_Handler.py* which contains the main utility class for dealing with all CSV file operations. It can build file paths, checks whether the required files already exist or not. It is also responsible for reading records from CSV, updating the already existing data in the CSV file, appending rows when necessary and deleting files.

Main Classes

- **Library** which serves as the main controller class, where books, users and borrowing records are managed.
- **Book** where each book´s information is stored, it is also responsible for handling availability of copies.

- **Users** which is responsible for storing information about users, it also does simple and basic email validations.
- **Borrowing_Records** and as the name goes, it pretty much deals with book borrowing records and data.
- **FileHandler** where CSV file operations are handled. It creates, reads, updates, writes and appends new rows.

Management of Input Data

The input data to the system are stored in CSV files, which means that when the program starts, the data is loaded from CSV files into the memory. When there is an input of new data, change or update, as well as deletion, it is executed back into the CSV files.

Implementation Overview

- This command-line based application was built using core Python features. As mentioned before, it uses an Object-oriented design, where Classes represent and arrange system components.
- Operations such as adding, borrowing and returning, etc. is functional through methods, which are dedicated for various functions.
- Validation check and exceptions are used for the management of invalid inputs and runtime issues. For example: wrong formats, duplicate entries or file errors.
- Data storage is implemented through CSV files.
- Readability was ensured through clear names, a style of code that could be viewed as “consistent” and methods with simple structure.
- System is split into different files, to make sure that the source code is not cluttered in one file, making it long and harder to read.

Reflection

The main challenge that affected the quality of my work and code was the time that I was given for such a big assignment, I wish that the assignment was given to me 1 month before deadline, due to the fact that this was not the only module where I had an assignment due as well. Another challenge that I have faced was proper documentation, because I made the mistake of not typing comments in the source code, thinking that I would be able to come back later before the deadline to add them, I also should have started typing the report about the project the moment I started typing the source code, this would have been so much better, because then I would be able to express my way of thinking at the moment that it happens instead of sitting for hours trying to remember the logic behind what I wrote. Another issue was the debugging, with how short of a time I had, I was not able to make sure that everything was 100% working, therefore a future improvement would be adding “pytest” to speed up the debugging process.

Another improvement that I would do is making a SQL database for the system, rather than just CSV files, as it is more reliable in general, due to what it offer from better data integrity to lower risk of file corruption. I